Graph-Based Movie Recommendation Using a User–Movie Bipartite Graph

Mirza Baig
mirza.baig@smu.ca
Saint Mary's University
Halifax, Nova Scotia, Canada

Tooba Javed
tooba.javed@smu.ca
Saint Mary's University
Halifax, Nova Scotia, Canada

Abstract

Traditional recommendation approaches such as matrix factorization model user–item interactions independently and do not exploit the relational structure present in the data. In this project, the MovieLens dataset is represented as a user–movie bipartite graph and movie recommendation is framed as a link prediction problem. Four models are implemented and compared: a popularity baseline, matrix factorization via SVD, GraphSAGE, and LightGCN. All models are evaluated using Precision@K, Recall@K, and NDCG@K on a chronological train/validation/test split with negative sampling. LightGCN achieves the best overall performance, reaching a Precision of 0.3083, Recall of 0.2445, and NDCG of 0.3090 at K=10, and continues to improve as K increases. Matrix factorization performs the worst despite achieving the highest Precision at K=1, with its Recall remaining flat at 0.0557 across all values of K. These results demonstrate that graph-based collaborative filtering with an appropriate training objective outperforms traditional approaches, and that architecture design is as important as access to the graph structure.

CCS Concepts

• **Information systems** → **Collaborative filtering**; • **Computing methodologies** → *Neural networks.*

Keywords

graph neural networks, recommender systems, link prediction, LightGCN, GraphSAGE, collaborative filtering, bipartite graph

1 Introduction

Recommender systems help users navigate large item catalogs by predicting which items they are most likely to enjoy. In movie platforms, users typically interact with only a small fraction of available titles, creating a sparse interaction space that makes preference prediction difficult. Standard approaches such as matrix factorization learn user and item representations from direct interactions but do not capture the relational structure that exists across the broader interaction graph.

Graph-based machine learning addresses this limitation by modeling users and items as nodes in a network, with edges encoding observed interactions. Graph neural networks (GNNs) can propagate information through these connections to learn embeddings that reflect both direct interactions and indirect, multi-hop collaborative signals [2, 4]. Prior work such as LightGCN [4] and PinSage [7] has demonstrated that this structural information can meaningfully improve recommendation quality over matrix factorization methods [6].

This project constructs a user–movie bipartite graph from the MovieLens dataset [3] and frames recommendation as a link prediction task. Two graph-based models (GraphSAGE and LightGCN) are compared against a popularity baseline and a matrix factorization baseline. The aim is to determine whether leveraging graph structure improves recommendation quality, and to understand what role model architecture plays in realizing that improvement.

2 Methodology

2.1 Dataset

The MovieLens Latest Small dataset [3] is used, which contains 100,836 ratings from 610 users across 9,742 movies. Each record includes a user ID, movie ID, a rating on a 1–5 scale, and a Unix timestamp. Figure 1 shows the distribution of rating values and ratings per user. Ratings of 4.0 are the most common, and the per-user distribution is heavily skewed: most users have submitted fewer than 100 ratings, while a small number have submitted several hundred or more. This sparsity motivates the use of graph-based methods that can propagate information across the interaction structure to compensate for limited direct observations per user.

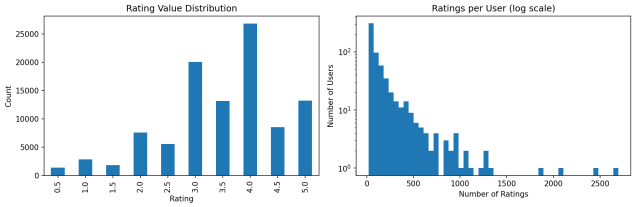

Figure 1: Left: distribution of rating values across all 100,836 interactions. Right: number of ratings per user on a log scale, illustrating the heavy-tailed, sparse nature of the dataset.

2.2 Graph Construction

The dataset is represented as a bipartite graph $\mathcal{G} = (\mathcal{U} \cup \mathcal{M}, \mathcal{E})$, where $\mathcal{U}$ and $\mathcal{M}$ are the sets of user and movie nodes respectively. An edge $(u, m) \in \mathcal{E}$ is added when user u rated movie m with a score of 4 or above, treating this as a positive interaction. Ratings below 4 are discarded. This binarization is standard practice in top-K recommendation evaluation [4]. The NetworkX library was used for graph construction. Figure 2 shows a subgraph of the top 20 most active users and top 15 most-rated movies, illustrating how highly active users connect to a dense shared core of popular movies while also branching out to less frequently rated titles.

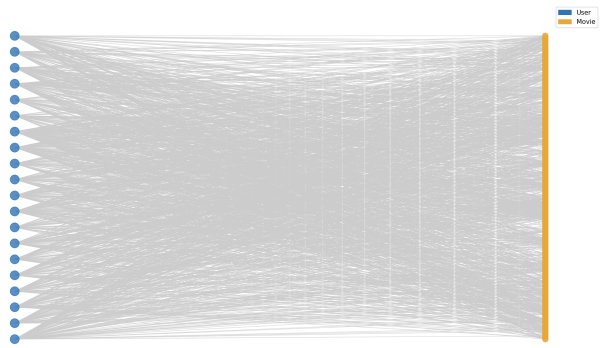

Figure 2: Bipartite subgraph of the top 20 users and top 15 movies by interaction count. Blue nodes are users; orange nodes are movies. The dense connectivity among this subset illustrates the collaborative signal available to graph-based models.

2.3 Data Preparation

Interactions were sorted by timestamp and split chronologically: 70% for training, 10% for validation, and 20% for testing. This ensures the model is always evaluated on interactions that occurred after training, reflecting a realistic deployment scenario. Because each interaction is assigned to exactly one split, every (user, movie) pair in the validation or test set is guaranteed not to appear in the training set. To handle the cold start problem—where users or movies appear in validation or test but not in training—those interactions were reassigned to the training set. After this adjustment, 48,580 positive interactions were retained. Table 1 shows the final split statistics and candidate pool sizes.

Table 1: Dataset statistics after chronological splitting and cold start resolution.

Split	Positive interactions	Candidate pairs
Training	47,746	—
Validation	378	19,278
Test	456	23,256

For evaluation, each positive interaction in the validation and test sets was paired with sampled negatives—movies the user had not interacted with—forming the candidate pools in Table 1. Models rank all candidates per user, and metrics are computed over those ranked lists.

2.4 Models

Popularity Baseline. Each movie is scored by the number of positive interactions it received in training. All users receive the same ranked list. This non-personalized baseline serves as a lower bound that any personalized model should exceed.

Matrix Factorization (SVD). SVD from the Surprise library [5] decomposes the user–item interaction matrix into user and item latent factor vectors by minimizing reconstruction error on binary training interactions. Inference scores are dot products of the corresponding vectors.

GraphSAGE. GraphSAGE [2] learns node embeddings by aggregating features from local neighborhoods. Each node is initialized with a learnable embedding, and a two-layer encoder applies mean neighborhood aggregation over the bipartite graph. User–movie scores are dot products of the resulting embeddings. Training uses binary cross-entropy loss. A learning rate sweep over $\{10^{-4}, 5{\times}10^{-4}, 10^{-3}, 3{\times}10^{-3}\}$ was first conducted at 100 epochs with hidden dimension 64, dropout 0.2, and weight decay 10^{-5}, selecting the best learning rate by validation Recall@10. A subsequent tuned search then extended training to 220 epochs with hidden dimension 128, dropout 0.1, negative ratio 3, and weight decay 10^{-6}. The final configuration used lr=10^{-3}, hidden dimension 128, dropout 0.1, 220 epochs, and 3 negatives per positive.

LightGCN. LightGCN [4] is designed specifically for collaborative filtering. It removes feature transformation and nonlinear activation from standard graph convolution, keeping only neighborhood aggregation. The final embedding for each node is the mean across all layer representations:

$$\mathbf{e}_u = \frac{1}{L+1} \sum_{l=0}^{L} \mathbf{e}_u^{(l)} \tag{1}$$

where L is the number of convolution layers. The `LGConv` operator from PyTorch Geometric [1] was used. Training was performed with Bayesian Personalized Ranking (BPR) loss, which directly optimizes the relative ranking of positive items above negatives. The same learning rate sweep was conducted as for GraphSAGE. A tuned hyperparameter search then explored four configurations varying embedding dimension (128, 192), number of layers (3, 4), epochs (220, 240), negative ratio (3, 4), and regularization weight (10^{-4}, 10^{-5}), again selecting by validation HR@10. The best configuration used lr=10^{-3}, embedding dimension 128, 3 convolution layers, negative ratio 3, regularization weight 10^{-4}, and 220 training epochs.

2.5 Evaluation Metrics

All models are evaluated on the test set at $K \in \{1, 5, 10, 15, 20\}$ using three standard top-K ranking metrics.

Precision@K is the fraction of the top-K items that are true positives: Precision@K = $|\{\text{relevant}\} \cap \{\text{top-}K\}|/K$.

Recall@K is the fraction of all positive items retrieved in the top K: Recall@K = $|\{\text{relevant}\} \cap \{\text{top-}K\}|/|\{\text{relevant}\}|$.

NDCG@K (Normalized Discounted Cumulative Gain) rewards correct recommendations appearing higher in the ranked list, normalized against the ideal ranking.

3 Results

Table 2 reports test set performance at K=10. Figure 3 visualizes those results, and Figure 4 shows how all three metrics evolve across $K \in \{1, 5, 10, 15, 20\}$.

At K=10, LightGCN leads on all three metrics. The popularity baseline ranks second with Precision of 0.2417 and NDCG of 0.2531. GraphSAGE falls below the popularity baseline on all three metrics at K=10. Matrix factorization scores the lowest Precision (0.0417) and NDCG (0.1160) at K=10, despite achieving the highest Precision

Table 2: Test set results at $K = 10$. Best values in bold.

Model	Precision@10	Recall@10	NDCG@10
Popularity	0.2417	0.1850	0.2531
Matrix Fact.	0.0417	0.0557	0.1160
GraphSAGE	0.1917	0.1366	0.2204
LightGCN	**0.3083**	**0.2445**	**0.3090**

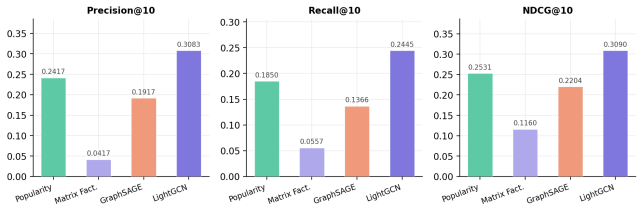

Figure 3: Precision@10, Recall@10, and NDCG@10 on the test set for all four models. LightGCN achieves the highest score on every metric; matrix factorization achieves the lowest.

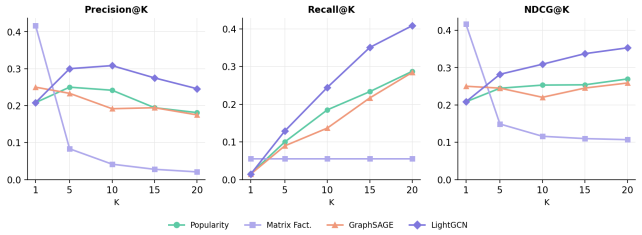

Figure 4: Precision@K, Recall@K, and NDCG@K as K varies from 1 to 20. LightGCN achieves the highest Recall and NDCG for $K \geq 5$, reaching Recall of 0.4086 at K=20. Matrix factorization Recall is flat at 0.0557 across all K.

of any model at K=1 (0.4167). Its Recall remains unchanged at 0.0557 across all five values of K, indicating that the model consistently retrieves the same single positive item per user regardless of list length.

As K grows, LightGCN's advantage in Recall becomes more pronounced, reaching 0.3507 at K=15 and 0.4086 at K=20, compared to 0.2874 for Popularity and 0.2845 for GraphSAGE at K=20. LightGCN also achieves the highest NDCG at every value of K from 5 onward, increasing to 0.3529 at K=20.

4 Discussion

LightGCN vs. GraphSAGE. Both models operate on the same bipartite graph with comparable hyperparameter budgets, yet LightGCN consistently outperforms GraphSAGE from K=5 onward. The key difference is the training objective: BPR loss directly optimizes the relative ranking of positive items above negatives, which aligns with how the models are evaluated. GraphSAGE uses point-wise binary cross-entropy loss, which does not directly optimize ranking. Removing nonlinear activations in LightGCN also avoids oversmoothing across 3 convolution layers, preserving discriminative signal in the final embeddings. These results confirm that access to graph structure is not on its own sufficient—the architecture and training objective must be designed around the recommendation task.

Multi-hop collaborative signal. With 3 convolution layers, each user embedding in LightGCN aggregates information from movies up to 3 hops away, capturing the preferences of users with similar taste profiles. This is the structural advantage graph-based methods offer over matrix factorization, which considers only direct user–item pairs.

Matrix factorization and flat Recall. The constant Recall of 0.0557 across all values of K indicates that for most users in the test set, SVD ranks exactly one positive item reliably high and places all remaining positives outside the top 20. This is consistent with the model collapsing to near-uniform scores for most candidates when interactions are binarized, making it unsuitable for generating a diverse top-K recommendation list despite its strong Precision at K=1.

Popularity as a strong baseline. The popularity baseline is competitive because popular movies tend to remain popular over time, making the heuristic naturally well-aligned with a chronological test set. This is a known property of temporal evaluation splits rather than evidence of genuine personalization quality.

Limitations. Node features such as movie genres and user-generated tags were not used. Incorporating them as initial embeddings could improve performance, particularly for users with few interactions. Uniform negative sampling may also overestimate model quality compared to harder sampling strategies that draw negatives from popular, plausible candidates.

5 Conclusion and Future Work

This project built a graph-based movie recommendation system on the MovieLens dataset, framing recommendation as link prediction on a user–movie bipartite graph. Across all evaluated values of K, LightGCN outperforms all other models on Recall and NDCG, improving over the popularity baseline by 27.6% in Precision at K=10 and achieving a Recall of 0.4086 at K=20. Matrix factorization collapses to flat Recall under binary interactions and chronological splitting, making it the weakest model despite a strong Precision at K=1. GraphSAGE demonstrates that using graph structure with a mismatched training objective is insufficient to compete with a model designed end-to-end for ranking.

These findings support the conclusion that graph-based collaborative filtering offers meaningful gains over traditional approaches, and that the training loss is a critical design choice in realizing those gains.

Future work could incorporate movie genre and tag metadata as initial node features, explore attention-based aggregation mechanisms, and investigate contrastive self-supervised training objectives that reduce reliance on negative sampling quality.

Acknowledgments

Source code for this project is available at https://github.com/toobajaved/GraphBasedMovieRecommendation.

References

[1] Matthias Fey and Jan Eric Lenssen. 2019. Fast Graph Representation Learning with PyTorch Geometric. In *ICLR Workshop on Representation Learning on Graphs and Manifolds*.

[2] William L. Hamilton, Zhitao Ying, and Jure Leskovec. 2017. Inductive Representation Learning on Large Graphs. In *Advances in Neural Information Processing Systems*, Vol. 30. Curran Associates.

[3] F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems* 5, 4 (2015), 1–19. doi:10.1145/2827872

[4] Xiangnan He, Kelong Deng, Xiang Wang, Yan Li, YongDong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '20)*. ACM, 639–648. doi:10.1145/3397271.3401063

[5] Nicolas Hug. 2020. Surprise: A Python Library for Recommender Systems. *Journal of Open Source Software* 5, 52 (2020), 2174. doi:10.21105/joss.02174

[6] Yehuda Koren, Robert Bell, and Chris Volinsky. 2009. Matrix Factorization Techniques for Recommender Systems. *Computer* 42, 8 (2009), 30–37. doi:10.1109/MC.2009.263

[7] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L. Hamilton, and Jure Leskovec. 2018. Graph Convolutional Neural Networks for Web-Scale Recommender Systems. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '18)*. ACM, 974–983. doi:10.1145/3219819.3219890